

GITHUB CITY

Project Documentation

Technical project overview and IBM Bob–driven development workflow

1. Project Overview

GitHub City is a full-stack 3D procedural city builder that turns a GitHub contribution calendar into a navigable 3D city. The project combines a React frontend, Three.js/React Three Fiber rendering, Tailwind CSS styling, and an Express backend that securely retrieves GitHub contribution data.

The visualization uses 371 contribution days (53 weeks × 7 days). Contribution counts determine the generated building class and height, turning contribution history into a city that can be explored by driving through it.

2. Project Goal

The goal is to make GitHub contribution activity explorable rather than purely graphical: the user generates a city from contribution activity and can drive through the resulting environment while interacting with camera, environment, audio, screenshot, and cinematic features.

3. Current Technology Stack

- Frontend: React 19 with TypeScript.
- 3D rendering: Three.js r184 and React Three Fiber.
- 3D utilities: @react-three/drei.
- Styling: Tailwind CSS v4.
- Backend: Node.js with Express 5.
- GitHub integration: GitHub GraphQL API through a server-side Express proxy.
- Build/development: Vite with TypeScript.
- Icons/UI support: Lucide React.

4. IBM Bob as the Primary Development Bot

IBM Bob is the primary AI development partner used for this project. Instead of using AI only as a code generator, the project uses Bob across the software development lifecycle: understanding the existing codebase, evaluating technical choices, planning work, implementing changes, debugging, and iterating on completed features.

IBM describes Bob as an AI Software Development Lifecycle (SDLC) partner. Its three core modes are designed for different development situations: Ask for understanding and analysis, Plan for architecture and implementation planning, and Agent for writing, modifying, refactoring, and executing development work.

For this project, these modes are treated as a controlled development pipeline rather than three unrelated chat options.

4.1 Why IBM Bob is the Primary Bot

- Codebase understanding: Bob can inspect the existing implementation before proposing changes, which is important for a project with interconnected React, Three.js, physics, UI, and Express systems.
- Technical decision support: Ask mode is used before committing to a framework, library, architecture, implementation strategy, or change with meaningful technical trade-offs.
- Structured planning: Plan mode converts requirements into an actionable implementation plan and breaks larger features into manageable tasks before code is changed.
- Implementation: Agent mode is the main execution stage for creating files, editing code, refactoring existing code, running commands, and completing planned development work.
- Iterative debugging: When implementation or builds fail, Bob can inspect the failure, propose a correction, apply the fix, and continue the development cycle.
- Context-aware development: Bob supports persistent project context through /init and AGENTS.md so the development assistant can retain project structure, conventions, and goals across conversations.
- Controlled execution: Bob presents tool actions for review and approval, allowing the project workflow to keep human oversight over file changes and command execution. [↗cite↗turn0search1↗turn0search15↗](#)

4.2 IBM Bob Modes Used in This Project

IBM Bob Mode	Project Purpose	Typical Use	Expected Output
Ask	Understand and evaluate before changing code	Compare technologies, inspect architecture, analyze an existing component, investigate a bug, evaluate a technical approach.	Technical explanation, recommendation, trade-off analysis, or diagnosis.
Plan	Design the implementation before coding	Break a feature into tasks, define affected files/components, identify dependencies, define implementation order and validation steps.	Actionable implementation plan and task breakdown.
Agent	Build and modify the project	Implement planned features, edit/refactor code, create files, run commands, fix build/runtime issues, and iterate.	Working code, updated files, executed checks, and an implementation summary.

4.3 IBM Bob Workflow for This Project

The project follows an

Ask → Plan → Review → Agent → Validate → Document → Iterate workflow

This keeps AI-assisted development structured and prevents direct implementation from becoming the default response to every requirement.

- 1. Requirement:** Define the feature, bug, improvement, or technical question in clear terms.
- 2. Ask:** Use Ask mode to inspect the relevant code and understand the current implementation. If a technology or architecture choice is involved, use Ask to compare options and recommend an approach.

3. Plan: Move to Plan mode for work that requires implementation. Bob analyzes the codebase, identifies affected files, decomposes the requirement into tasks, and defines the implementation sequence.

4. Human Review: Review the plan before implementation. Confirm scope, affected systems, constraints, and whether the proposed change preserves existing functionality.

5. Agent: Switch to Agent mode and instruct Bob to implement the approved plan. Bob reads the required files, edits or creates code, and executes necessary development commands with the configured approval controls.

6. Validate: Run the relevant build, lint, runtime, and feature checks. If something fails, use Bob to diagnose and correct the issue rather than bypassing the failure.

7. Document: Update project documentation such as roadmap.md and dev-log.md when the completed work changes project status or records a meaningful implementation milestone.

8. Iterate: If the result is incomplete or needs refinement, return to Ask or Plan for clarification and then use Agent for the next implementation pass.

4.4 Example IBM Bob Workflow in the Project

Example: adding a new city feature such as repository-based landmarks.

- Ask: inspect the current city-generation architecture and determine how landmark data should fit into the existing contribution-grid model.
- Ask: compare possible data structures or rendering approaches and identify performance implications for the existing Three.js scene.
- Plan: define the required data flow, affected React/Three.js files, landmark generation rules, UI changes, and validation steps.
- Review: confirm that the plan does not unnecessarily rewrite the current city generator or vehicle system.
- Agent: implement the approved changes, create or update the necessary files, and run the relevant build/check commands.
- Validate: test the new landmarks together with driving, camera, day/night, and existing city generation.
- Document: record the completed work in the roadmap/development log where appropriate.
- Iterate: use Bob again to fix defects or refine the feature based on observed behavior.

4.5 Development Rules When Using IBM Bob

- Inspect the existing implementation before making changes.
- Prefer the smallest change that satisfies the requirement.
- Do not replace working systems without a clear technical reason.
- Keep existing controls, visuals, physics, and data behavior intact unless the requirement explicitly changes them.
- Use Ask for understanding, Plan for design, and Agent for implementation.
- Review plans before large or cross-cutting implementation work.
- Validate changes instead of assuming generated code is correct.
- Keep roadmap.md and dev-log.md aligned with actual project progress.

5. User Experience

5.1 Input and Data Source

The application supports two data modes: Real GitHub Data and Demo Data. In Real GitHub Data mode, the user provides a GitHub username. Demo mode can generate the city without live GitHub access.

5.2 Main Experience

Enter/select the GitHub username and data source.

Fetch or load contribution data.

Map contribution data to the 3D grid.

Generate the procedural city.

Enter the driveable 3D environment.

Use driving, camera, environment, audio, screenshot, and cinematic controls.

6. GitHub Data Architecture

Live GitHub data is retrieved through the Express backend rather than directly exposing the GitHub credential to the browser. The backend exposes `/api/github/:username`, calls the GitHub GraphQL API, normalizes the contribution calendar, calculates streak statistics, and returns the data required by the frontend.

Flow: Username → Express backend → GitHub GraphQL API → normalized contribution grid → 3D city.

6.1 Server-side Token

The backend reads `GITHUB_TOKEN` from the environment. The repository's `.gitignore` excludes `.env`, so the credential is not intended to be committed to the repository.

The token is used by the backend to authorize GitHub GraphQL requests. The frontend only supplies the username to the backend endpoint.

6.2 Backend Processing

- Checks the in-memory cache before making a new GitHub request.
- Calls `https://api.github.com/graphql` with the server-side token.
- Retrieves the user's contribution calendar.
- Normalizes the data to 371 days.
- Calculates total contributions, current streak, and longest streak.
- Maps each day to x/z grid coordinates.
- Returns `isMock`: false for live data.
- Caches successful responses for 10 minutes.
- Returns explicit errors for missing/invalid tokens, rate limits, missing users, unavailable contribution data, and API failures.

7. Procedural City Generation

The city is centered at [0, 0, 0] and uses a 53 × 7 contribution grid. The layout is organized into city blocks with a central avenue, north-south streets, sidewalks, streetlights, and procedural building variations.

The current implementation also includes class-specific visual details such as rooftop HVAC/detailing, neon facade elements, tower tiers, antennae, and warning beacons.

8. Driving System

The project contains a low-poly sports car with kinematic movement, acceleration/deceleration, steering, sliding friction, hard braking, wheel rolling, and front-wheel steering.

8.1 Collision and Boundary

- Building collisions use AABB-based sliding behavior so the vehicle can glide along building walls. An invisible platform boundary also limits the vehicle to the playable area; the README documents the current safety limit as 76 units.

8.2 Camera

- Driving mode uses a third-person follow camera with user-controlled yaw/pitch and adjustable distance. The camera smoothly recenters after inactivity and includes a collision-aware distance adjustment.

9. Environment System

9.1 Day/Night

- The environment supports day/night transitions. Lighting, fog, building window appearance, and vehicle lights respond to the selected time of day.

9.2 Daytime Sky and Clouds

- Custom gradient sky dome.
- Blue daytime sky gradient.
- Procedurally generated low-poly cloud clusters.
- Clouds drift horizontally and wrap around the city.
- Cloud visibility transitions with day/night state.

9.3 Night Sky

- Night mode includes a star field with dynamically rendered particles and illuminated building windows.

9.4 Environment Toggles

- The project roadmap records settings toggles for clouds, fog, and stars.

10. Mobile Support

The project includes mobile/tablet detection for touch devices and provides a virtual joystick and handbrake control. The interface also reduces visual clutter on smaller screens.

11. Security and Configuration

- GITHUB_TOKEN is a backend environment variable.
- .env is excluded in .gitignore.
- The token is not intended to be exposed to frontend assets.
- The backend handles GitHub authentication and API requests.
- API errors are converted into user-facing error categories rather than returning credentials.

12. Development Principles

- Do not unnecessarily rewrite existing working code.
- Inspect the existing implementation before making a change.
- Make the smallest change required for the requested feature.
- Preserve existing game behavior unless the requirement explicitly changes it.
- Test the changed system and check related systems for regressions.
- Keep roadmap.md and dev-log.md current.

Final Project Description

GitHub City is a browser-based, full-stack 3D visualization that transforms a developer's GitHub contribution calendar into a procedural city. The city is generated from 371 contribution days, building classes are determined by contribution counts, and the user can drive through the generated environment. The current project also includes live GitHub GraphQL integration through a secure Express proxy, a demo-data path, vehicle collisions and boundaries, controllable cameras, day/night visuals, clouds, stars, fog, screenshots, cinematic camera paths, and ambient audio.

IBM Bob is used as the project's development workflow: Ask for technology and architecture decisions, Plan for feature decomposition and implementation planning, and Agent for final construction using the selected stack.

The project-specific technical content is grounded in the provided project repository and its README, package configuration, server implementation, roadmap, development log, source structure, and configuration files. IBM Bob workflow details are based on IBM's official Bob documentation.

Source Basis

This document was prepared from the current public repository the project repository, including its README.md, package.json, server.js, roadmap.md, dev-log.md, .gitignore, and current src structure.

IBM Bob Reference

IBM Bob documentation describes Ask, Plan, and Agent as specialized modes for understanding, planning, and implementation, with approval-controlled tool execution and persistent project context through AGENTS.md. These capabilities are incorporated into the project's primary development workflow.

Official IBM Bob documentation: <https://bob.ibm.com/docs/ide>